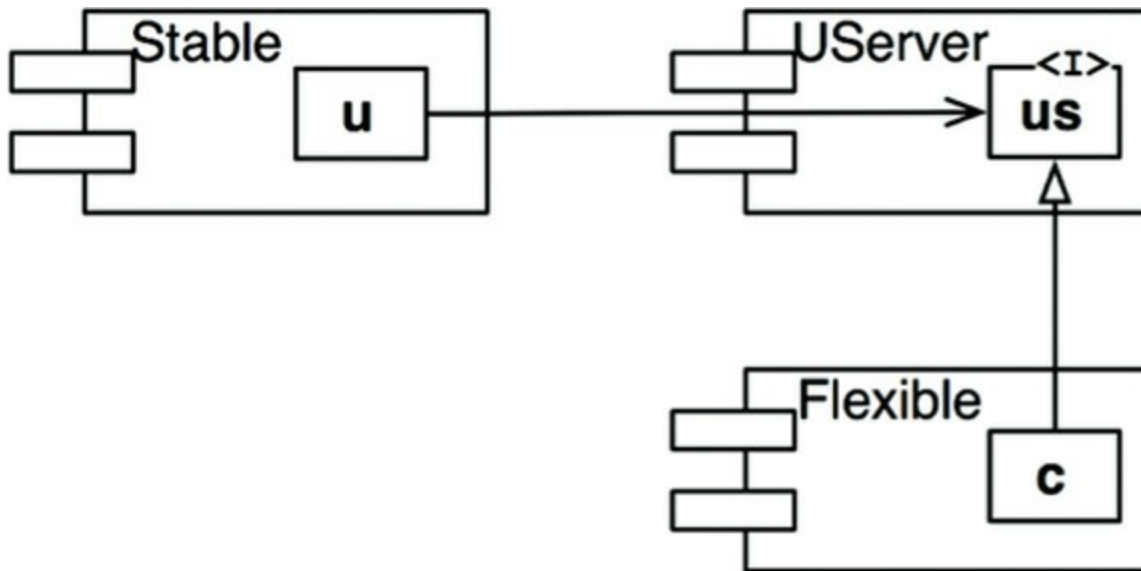


components to depend on `UService`. `UService` is very stable ( $I = 0$ ), and `Flexible` retains its necessary instability ( $I = 1$ ). All the dependencies now flow in the direction of *decreasing*  $I$ .



**Figure 14.11** `c` implements the interface class `us`

## Abstract Components

You may find it strange that we would create a component—in this example, `UService`—that contains nothing but an interface. Such a component contains no executable code! It turns out, however, that this is a very common, and necessary, tactic when using statically typed languages like Java and C#. These abstract components are very stable and, therefore, are ideal targets for less stable components to depend on.

When using dynamically typed languages like Ruby and Python, these abstract components don't exist at all, nor do the dependencies that would have targeted them. Dependency structures in these languages are much simpler because dependency inversion does not require either the declaration or the inheritance of interfaces.

# THE STABLE ABSTRACTIONS PRINCIPLE

*A component should be as abstract as it is stable.*

## WHERE DO WE PUT THE HIGH-LEVEL POLICY?

Some software in the system should not change very often. This software represents high-level architecture and policy decisions. We don't want these business and architectural decisions to be volatile. Thus the software that encapsulates the high-level policies of the system should be placed into stable components ( $I = 0$ ). Unstable components ( $I = 1$ ) should contain only the software that is volatile—software that we want to be able to quickly and easily change.

However, if the high-level policies are placed into stable components, then the source code that represents those policies will be difficult to change. This could make the overall architecture inflexible. How can a component that is maximally stable ( $I = 0$ ) be flexible enough to withstand change? The answer is found in the OCP. This principle tells us that it is possible and desirable to create classes that are flexible enough to be extended without requiring modification. Which kind of classes conform to this principle? *Abstract* classes.

## INTRODUCING THE STABLE ABSTRACTIONS PRINCIPLE

The Stable Abstractions Principle (SAP) sets up a relationship between stability and abstractness. On the one hand, it says that a stable component should also be abstract so that its stability does not prevent it from being extended. On the other hand, it says that an unstable component should be concrete since its instability allows the concrete code within it to be easily changed.

Thus, if a component is to be stable, it should consist of interfaces and abstract classes so that it can be extended. Stable components that are extensible are flexible and do not overly constrain the architecture.

The SAP and the SDP combined amount to the DIP for components. This is true because the SDP says that dependencies should run in the direction of stability, and the SAP says that stability implies abstraction. Thus *dependencies run in the direction of abstraction*.

The DIP, however, is a principle that deals with classes—and with classes there are no shades of gray. Either a class is abstract or it is not. The combination of the SDP and the SAP deals with components, and allows that a component can be partially abstract and partially stable.

## MEASURING ABSTRACTION

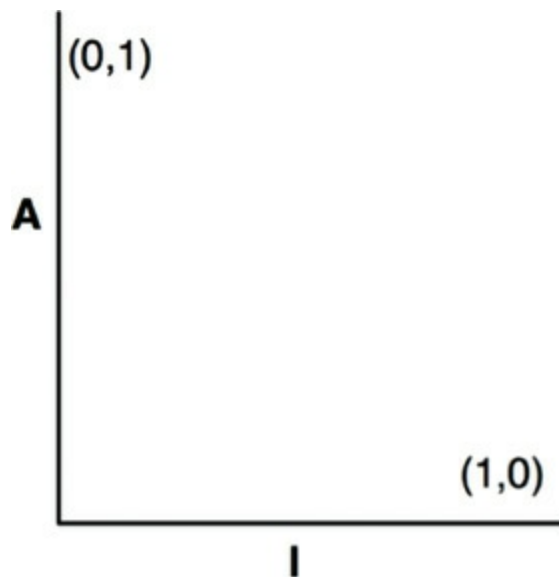
The  $A$  metric is a measure of the abstractness of a component. Its value is simply the ratio of interfaces and abstract classes in a component to the total number of classes in the component.

- $N_c$ : The number of classes in the component.
- $N_a$ : The number of abstract classes and interfaces in the component.
- $A$ : Abstractness.  $A = N_a \div N_c$ .

The  $A$  metric ranges from 0 to 1. A value of 0 implies that the component has no abstract classes at all. A value of 1 implies that the component contains nothing but abstract classes.

## THE MAIN SEQUENCE

We are now in a position to define the relationship between stability ( $I$ ) and abstractness ( $A$ ). To do so, we create a graph with  $A$  on the vertical axis and  $I$  on the horizontal axis ([Figure 14.12](#)). If we plot the two “good” kinds of components on this graph, we will find the components that are maximally stable and abstract at the upper left at  $(0, 1)$ . The components that are maximally unstable and concrete are at the lower right at  $(1, 0)$ .



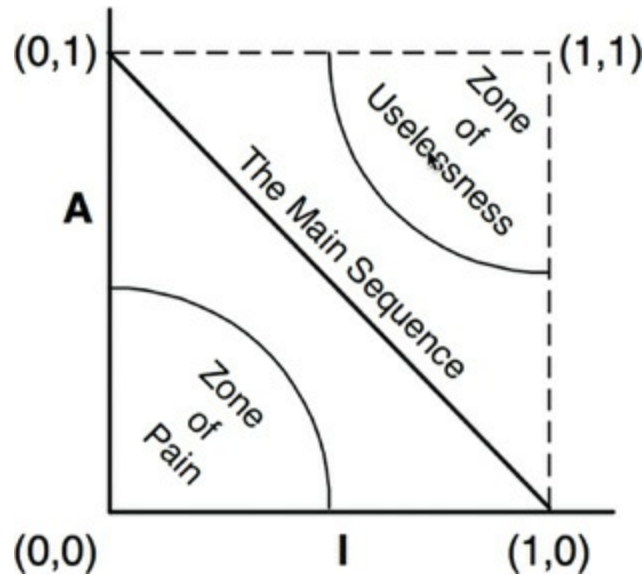
---

**Figure 14.12** The I/A graph

Not all components fall into one of these two positions, because components often have *degrees* of abstraction and stability. For example, it is very common for one abstract class to derive from another abstract class. The derivative is an abstraction

that has a dependency. Thus, though it is maximally abstract, it will not be maximally stable. Its dependency will decrease its stability.

Since we cannot enforce a rule that all components sit at either  $(0, 1)$  or  $(1, 0)$ , we must assume that there is a locus of points on the  $A/I$  graph that defines reasonable positions for components. We can infer what that locus is by finding the areas where components should *not* be—in other words, by determining the zones of *exclusion* (Figure 11.13).



**Figure 14.13** Zones of exclusion

### The Zone of Pain

Consider a component in the area of  $(0, 0)$ . This is a highly stable and concrete component. Such a component is not desirable because it is rigid. It cannot be extended because it is not abstract, and it is very difficult to change because of its stability. Thus we do not normally expect to see well-designed components sitting near  $(0, 0)$ . The area around  $(0, 0)$  is a zone of exclusion called the *Zone of Pain*.

Some software entities do, in fact, fall within the Zone of Pain. An example would be a database schema. Database schemas are notoriously volatile, extremely concrete, and highly depended on. This is one reason why the interface between OO applications and databases is so difficult to manage, and why schema updates are generally painful.

Another example of software that sits near the area of  $(0, 0)$  is a concrete utility library. Although such a library has an  $I$  metric of 1, it may actually be nonvolatile.

Consider the `String` component, for example. Even though all the classes within it are concrete, it is so commonly used that changing it would create chaos. Therefore `String` is nonvolatile.

Nonvolatile components are harmless in the (0, 0) zone since they are not likely to be changed. For that reason, it is only volatile software components that are problematic in the Zone of Pain. The more volatile a component in the Zone of Pain, the more “painful” it is. Indeed, we might consider volatility to be a third axis of the graph. With this understanding, [Figure 14.13](#) shows only the most painful plane, where volatility = 1.

### **The Zone of Uselessness**

Consider a component near (1, 1). This location is undesirable because it is maximally abstract, yet has no dependents. Such components are useless. Thus this area is called the *Zone of Uselessness*.

The software entities that inhabit this region are a kind of detritus. They are often leftover abstract classes that no one ever implemented. We find them in systems from time to time, sitting in the code base, unused.

A component that has a position deep within the Zone of Uselessness must contain a significant fraction of such entities. Clearly, the presence of such useless entities is undesirable.

## **AVOIDING THE ZONES OF EXCLUSION**

It seems clear that our most volatile components should be kept as far from both zones of exclusion as possible. The locus of points that are maximally distant from each zone is the line that connects (1, 0) and (0, 1). I call this line the *Main Sequence*.<sup>2</sup>

A component that sits on the Main Sequence is not “too abstract” for its stability, nor is it “too unstable” for its abstractness. It is neither useless nor particularly painful. It is depended on to the extent that it is abstract, and it depends on others to the extent that it is concrete.

The most desirable position for a component is at one of the two endpoints of the Main Sequence. Good architects strive to position the majority of their components at those endpoints. However, in my experience, some small fraction of the

components in a large system are neither perfectly abstract nor perfectly stable. Those components have the best characteristics if they are on, *or close*, to the Main Sequence.

## DISTANCE FROM THE MAIN SEQUENCE

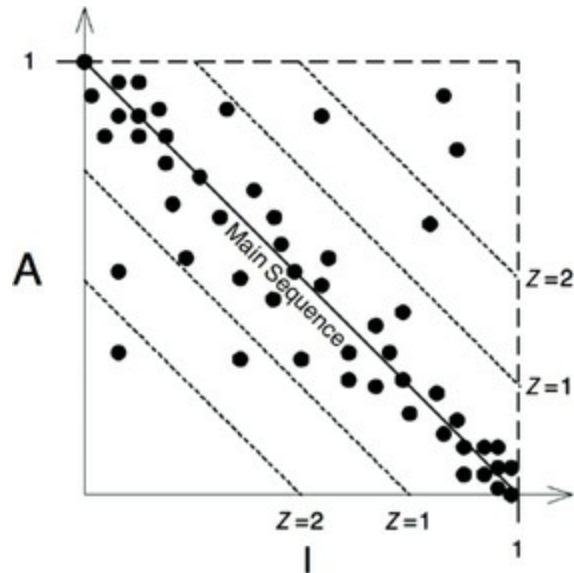
This leads us to our last metric. If it is desirable for components to be on, or close, to the Main Sequence, then we can create a metric that measures how far away a component is from this ideal.

- $D^3$ : Distance.  $D = |A+I-1|$ . The range of this metric is  $[0, 1]$ . A value of 0 indicates that the component is directly on the Main Sequence. A value of 1 indicates that the component is as far away as possible from the Main Sequence.

Given this metric, a design can be analyzed for its overall conformance to the Main Sequence. The  $D$  metric for each component can be calculated. Any component that has a  $D$  value that is not near zero can be reexamined and restructured.

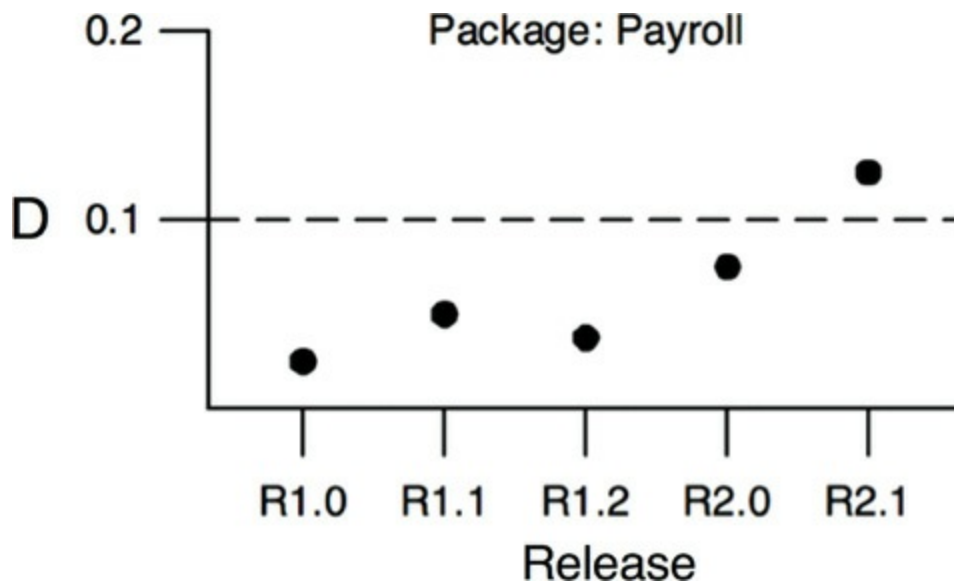
Statistical analysis of a design is also possible. We can calculate the mean and variance of all the  $D$  metrics for the components within a design. We would expect a conforming design to have a mean and variance that are close to zero. The variance can be used to establish “control limits” so as to identify components that are “exceptional” in comparison to all the others.

In the scatterplot in [Figure 14.14](#), we see that the bulk of the components lie along the Main Sequence, but some of them are more than one standard deviation ( $Z = 1$ ) away from the mean. These aberrant components are worth examining more closely. For some reason, they are either very abstract with few dependents or very concrete with many dependents.



**Figure 14.14** Scatterplot of the components

Another way to use the metrics is to plot the  $D$  metric of each component over time. The graph in [Figure 14.15](#) is a mock-up of such a plot. You can see that some strange dependencies have been creeping into the `Payroll` component over the last few releases. The plot shows a control threshold at  $D = 0.1$ . The R2.1 point has exceeded this control limit, so it would be worth our while to find out why this component is so far from the main sequence.



**Figure 14.15** Plot of  $D$  for a single component over time

## CONCLUSION

The *dependency management metrics* described in this chapter measure the conformance of a design to a pattern of dependency and abstraction that I think is a “good” pattern. Experience has shown that certain dependencies are good and others are bad. This pattern reflects that experience. However, a metric is not a god; it is merely a measurement against an arbitrary standard. These metrics are imperfect, at best, but it is my hope that you find them useful.

1. In previous publications, I used the names *Efferent* and *Afferent* couplings (Ce and Ca) for *Fan-out* and *Fan-in*, respectively. That was just hubris on my part: I liked the metaphor of the central nervous system.
2. The author begs the reader’s indulgence for the arrogance of borrowing such an important term from astronomy.
3. In previous publications, I called this metric  $D'$ . I see no reason to continue that practice.